

Interaktive Skripte mit KI erstellen

Werkstatt-Skript zum Workshop am 11.05.2026, TH Köln

Prof. Dr. Roman Bartnik

11.05.2026

Inhaltsverzeichnis

1. Willkommen	5
2. Was du heute baust	6
2.1. Drei Referenzen	6
2.1.1. Lakens — Statistical Inferences	6
2.1.2. Bartnik — KI als Hilfe zum Lehren und Lernen	6
2.1.3. Békés — Data Analysis with AI	6
2.2. Wähle deinen Beispielstrang	7
2.3. Tagesablauf	7
2.4. Wie du dieses Skript benutzt	7
2.5. Ordnerstruktur deines Skripts	8
2.6. Voraussetzungen	8
2.7. Quellen	8
 I. Teil 1 — Werkstatt	 9
3. 1 — Warum interaktiv?	10
3.1. Drei Argumente in drei Sätzen	10
3.1.1. Generatives Lernen	10
3.1.2. Retrieval Practice	10
3.1.3. Multimedia-Prinzip	10
3.2. Was das für dein Skript heißt	11
3.3. Wann kein interaktives Element?	11
3.4. Mach mit — Lernspiel zur Selbstprüfung	11
3.5. Tutor	12
 4. 2 — Bausteine und Workflow	 13
4.1. Worum es im Kern geht	13
4.2. Die sechs Bausteine — Lay-Erklärung zuerst	13
4.3. Der Workflow in einem Bild	14
4.4. RStudio als Arbeitszentrale — vier Bereiche	14
4.5. Mach mit — die ersten 30 Minuten	15
4.5.1. Schritt 1 — neues Projekt in RStudio	15
4.5.2. Schritt 2 — Render und Preview (in RStudio)	15
4.5.3. Schritt 3 — Beispielkapitel auf deinen Strang anpassen	15
4.5.4. Schritt 4 — Ordnerstruktur anlegen (in RStudio)	16
4.5.5. Schritt 5 — GitHub Desktop verknüpfen	16
4.5.6. Schritt 6 — GitHub Pages aktivieren	16
4.6. Tutor	17
 5. 3a — Inhaltsverzeichnis und Literatur	 18
5.1. Worum es im Kern geht	18
5.2. Inhaltsverzeichnis — drei Zeilen	18

5.3.	Zotero anbinden	18
5.3.1.	Mach mit — in fünf Schritten	19
5.3.2.	Zitieren	19
5.4.	Tutor	20
6.	3b — TH-Köln-Stil einbauen	21
6.1.	Worum es im Kern geht	21
6.2.	Wie Quarto Aussehen handhabt	21
6.3.	Die TH-Köln-Palette in Stichworten	21
6.4.	Mach mit — <code>_brand.yml</code> kopieren	22
6.5.	SCSS-Ergänzungen	22
6.6.	Logo einbinden	23
6.7.	Tutor	23
7.	4 — Interaktive Diagramme	24
7.1.	Worum es im Kern geht	24
7.2.	Drei Wege, ein Diagramm einzubauen	24
7.3.	Mach mit — ein interaktives Diagramm in 10 Minuten	24
7.4.	Wann kein interaktives Diagramm?	26
7.5.	Tutor	27
8.	5 — iFrames, Spiele und KI-Tutor	28
8.1.	Worum es im Kern geht	28
8.2.	iFrame in einer Zeile	28
8.3.	Drei Wege, ein Spiel einzubauen	28
8.4.	KI-Tutor als Link	29
8.4.1.	Option 1 — Custom GPT auf ChatGPT (heute genutzt)	29
8.4.2.	Option 2 — Custom GPT in der AcademicCloud	29
8.4.3.	Option 3 — eigenes statisches Widget oder voll gehosteter Tutor	30
8.5.	Wann kein KI-Tutor?	30
8.6.	Tutor	31
9.	6 — Slides mit Quarto (optional)	32
9.1.	Schnellbau in vier Schritten	32
9.2.	TH-Köln-Brand auf Slides	32
9.3.	Tutor	33
II.	Teil 2 — Quick-Starter	34
10.	Quick-Starter Guide	35
10.1.	Checkliste vor dem Workshop	35
10.2.	RStudio-Cockpit auf einen Blick	35
10.3.	Schritt-für-Schritt: dein erstes Skript	36
10.3.1.	1. Neues Quarto-Buch anlegen	36
10.3.2.	2. Erste Vorschau	36
10.3.3.	3. <code>_quarto.yml</code> minimal anpassen	36
10.3.4.	4. Ordner für Bilder, Daten und Widgets anlegen	37
10.3.5.	5. Erstes Kapitel schreiben	37
10.3.6.	6. Render	37
10.3.7.	7. GitHub Desktop: Repo anlegen	37
10.3.8.	8. GitHub Pages aktivieren	37

10.3.9. 9. Erste Änderung publizieren	37
10.4. Was als Nächstes?	38
11. Trouble-Shooting	39
11.1. 1. YAML-Fehler beim Rendern	39
11.2. 2. GitHub Pages zeigt 404	39
11.3. 3. docs/ wird nicht committet	39
11.4. 4. Zotero-Zitat erscheint als [?]	39
11.5. 5. iFrame ist leer	39
11.6. 6. CSV lässt sich nicht laden	40
11.7. 7. R-Paket fehlt	40
11.8. 8. Quarto-Version zu alt	40
11.9. 9. GitHub Desktop fragt immer nach Anmeldung	40
11.10. 10. Logo wird nicht angezeigt	40
11.11. Wenn nichts hilft — Tutor fragen	40
12. KI-Tutor: Link und System-Prompt	41
12.1. Der vorbereitete Workshop-Tutor (sofort nutzbar)	41
12.2. Eigenen Tutor anlegen — drei Hosting-Wege	41
12.2.1. Weg 1 — eigenes Custom GPT auf ChatGPT	41
12.2.2. Weg 2 — AcademicCloud (Hochschul-Login, EU-Datenhaltung)	42
12.2.3. Weg 3 — statisches Widget (nur für Fortgeschrittene)	42
12.3. System-Prompt (Copy-Paste)	42
12.4. Drei Kurz-Prompts für typische Aufgaben	43
12.4.1. A — Fehler erklären	43
12.4.2. B — Code reviewen	43
12.4.3. C — Kapitel ergänzen	43
13. Literatur	44

1. Willkommen

Skript zum Workshop am 11.05.2026

2. Was du heute baust

Am Ende des Tages steht ein eigenes interaktives Skript live im Netz. Du hast es selbst gerendert, gepusht und veröffentlicht. Das ist das Ziel — alles andere ist Mittel zum Zweck.

2.0.0.1. Am Ende des Tages kannst du ...

1. Ein leicht pflegbares **Online-Skript erstellen** (ein Quarto-Projekt lokal in RStudio anlegen, rendern und über GitHub Pages veröffentlichen),
2. ...das verschiedene **Formatierungen automatisch erzeugt** (etwa ein automatisches Inhaltsverzeichnis erzeugen und Quellen aus Zotero zitieren),
3. ...**Stilelemente** an Deine Vorlieben **anpasst** (das Skript im TH-Köln-Stil formatieren (`_brand.yml`, CSS)),
4. ...und den Lernenden erlaubt, mit den Inhalten zu **interagieren** (ein interaktives Diagramm und ein iFrame-Spiel einbauen),
5. ...und **Hilfe durch KI** bereitzustellen (einen KI-Tutor verlinken, der die Lernenden im Skript begleitet).

2.1. Drei Referenzen

Bevor wir bauen, schauen wir, was möglich ist. Drei Skripte zeigen die Bandbreite — vom Kapitel zum Buch zum Kurs.

2.1.1. Lakens — Statistical Inferences

Ein vollständiges Open-Access-Lehrbuch zur Statistik mit Quizzes, Code-Demos und Videos. Maßstab für „so gut kann ein Skript werden“.

[Zum Skript →](#)

2.1.2. Bartnik — KI als Hilfe zum Lehren und Lernen

Mein eigenes Skript, mit interaktiven Widgets, eingebetteten Tutor-Bots und H5P-Quizzes.

[Zum Skript →](#)

2.1.3. Békés — Data Analysis with AI

Ein vollständiger Kurs samt Folien, Notebooks und Übungen. Maßstab für „Skript plus Slides plus Code“.

[Zum Skript →](#)

2.2. Wähle deinen Beispielstrang

Damit du nicht auch noch Inhalte erfinden musst, nutzen wir ein durchgehendes Mini-Thema: eine **kurze Einführung ins wissenschaftliche Arbeiten**. Wähle einen der drei Stränge und behalte ihn den ganzen Tag bei.

Strang	Worum es geht	Quelle
A — Wissenschaftliche Einstellung	Skepsis, Belege, intellektuelle Demut	Calling Bullshit, Kap. „Wissenschaftliche Einstellung“
B — Recherche und Quellenqualität	Suchen, prüfen, Quellen einordnen	Calling Bullshit, Kap. „Kritische Informationssuche“
C — Transparent schreiben	Methoden offenlegen, Reproduzierbarkeit	Calling Bullshit, Kap. „Transparent schreiben“

Die Wahl ist Geschmackssache. Die technischen Schritte sind in jedem Strang identisch.

2.3. Tagesablauf

Uhrzeit	Block	Was passiert
09:00–09:45	Auftakt	Vorstellung, Vorerfahrung, Ziele, Tour durch die drei Vorbilder, Strangwahl
09:45–10:30	Kap. 1 — Didaktik	Drei Lerntheorie-Argumente, Lernspiel zur Selbstprüfung
10:30–11:45	Kap. 2 — Setup	Bausteine, Memory-Spiel, lokale Grundstruktur, erster Push live
11:45–12:30	Pause	
12:30–13:15	Kap. 3a — Übersicht	Inhaltsverzeichnis, Zotero-Bibliothek, automatische Zitate
13:15–14:00	Kap. 3b — Stil	TH-Köln-Brand einbauen
14:00–14:45	Kap. 4 — Charts	Interaktives Diagramm mit echten Daten
14:45–15:30	Kap. 5 — iFrames + Tutor	H5P-/iFrame-Einbettung, Link zum KI-Tutor
15:30–16:00	Showcase + Puffer	Wer früh durch ist, baut Slides (Kap. 6); alle anderen holen auf

2.4. Wie du dieses Skript benutzt

Jedes Kapitel öffnet mit einem Lernziel-Kasten („Was du nach diesem Kapitel kannst“), gefolgt von einem „**Mach mit**“-Block — das ist die eigentliche Übung. Dazwischen steht so wenig Text

wie möglich. Am Ende jedes Kapitels findest du einen Trouble-Shooting-Kasten („Was, wenn ...“) und einen fertigen **Tutor-Prompt** zum Kopieren.

💡 RStudio ist deine Arbeitszentrale

Innerhalb von RStudio hast du **Editor**, **Terminal** (für `quarto render` und `quarto preview`), **Files-Pane** (zum Anlegen von Ordnern und Dateien), **Build-Pane** (Knopf „Render Book“) und **Console** (für R-Pakete) auf einem Schirm. Daneben brauchst du nur **GitHub Desktop** zum Versionieren und einen Browser-Tab mit dem **Tutor**.

[Tutor öffnen](#) — „Interaktive Skripte“ →

2.5. Ordnerstruktur deines Skripts

Damit du immer weißt, wo etwas hingehört (Lakens-Konvention):

Ordner	Was kommt rein?
parts/	Deine Kapiteldateien (.qmd)
images/	Logos, Screenshots, statische Grafiken (PNG, JPG, SVG)
interactions/	HTML-Widgets ohne Backend (Lernspiel, Memory, Quiz)
include/	Wiederverwendbare qmd-Schnipsel (per <code>{{< include >}}</code>)
data/	CSV/JSON-Daten für interaktive Charts
appendix/	Quickstart, Troubleshooting, Tutor-Prompt

2.6. Voraussetzungen

Wenn du noch nichts installiert hast, springe zuerst in den [Quick-Starter Guide](#). Dort steht, was du in welcher Reihenfolge installierst (Quarto, R, RStudio, GitHub Desktop, Zotero) — mit Screenshots.

2.7. Quellen

Lakens (2022); Bartnik (2025); Békés (2025); Biggs & Tang (2011); Mayer (2021).

Teil I.

Teil 1 — Werkstatt

3. 1 — Warum interaktiv?

3.0.0.1. Was du nach diesem Kapitel kannst

- drei lerntheoretisch gestützte Argumente nennen, **warum** ein interaktives Skript ein passives Skriptum schlägt,
- jedes Argument einem **konkreten Skript-Element** zuordnen (Quiz, Diagramm, Tutor),
- entscheiden, **wann sich Interaktivität lohnt** und wann nicht.

3.1. Drei Argumente in drei Sätzen

Lehrforschung sagt es seit Jahren — aber selten so kompakt: Lernen klappt besser, wenn Lernende **selbst etwas tun, regelmäßig abrufen** und **Bilder neben Worten** bekommen. Drei Prinzipien, drei Belege, drei Konsequenzen für dein Skript.

3.1.1. Generatives Lernen

Wer den Stoff selbst zusammenfasst, erklärt oder anwendet, behält ihn besser, als wer ihn nur liest ([Fiorella & Mayer, 2016](#)). Das ist kein Geschmack, das ist eine Meta-Analyse über acht Lernaktivitäten — Zusammenfassen, Selbsterklären, Lehren, Mappen, Vorhersagen, Vorstellen, Zeichnen, Aufführen. Übersetzt: in deinem Skript braucht jedes Kapitel **eine Aufgabe, die der Lernende selbst tut**, nicht nur einen Text, den er liest.

3.1.2. Retrieval Practice

Sich an etwas erinnern zu **müssen** verbessert das Behalten stärker als wiederholtes Lesen ([Roediger & Karpicke, 2006](#)). Karpicke und Roediger ließen Studierende dasselbe Material drei Mal lesen oder zwei Mal lesen plus einmal aus dem Gedächtnis aufschreiben — die zweite Gruppe schlug die erste eine Woche später deutlich. Übersetzt: ein **kurzes Quiz** oder eine **Drag-and-Drop-Aufgabe** in deinem Skript wirkt wie eine Mini-Klausur, ohne dass jemand die Klausur korrigieren muss.

3.1.3. Multimedia-Prinzip

Worte plus passende Bilder schlagen Worte allein ([Mayer, 2021](#)). „Passend“ heißt: das Bild zeigt, wovon der Text spricht — nicht zur Dekoration, sondern als zweite Repräsentation desselben Inhalts. Übersetzt: das **interaktive Diagramm**, das du in Kapitel 4 einbauen wirst, hat einen lerntheoretischen Grund — keine Spielerei.

3.2. Was das für dein Skript heißt

Drei Bauteile, die du heute alle einbaust, lassen sich genau diesen drei Prinzipien zuordnen:

Prinzip	Skript-Element	Wo im Workshop?
Generatives Lernen	KI-Tutor, eigene Übungsaufgabe	Kap. 5
Retrieval Practice	Lernspiel, Memory, Quiz (iFrame oder H5P)	Kap. 1, 2, 5
Multimedia	Interaktives Diagramm, Grafiken	Kap. 4

Das ist die didaktische Architektur deines Skripts — und gleichzeitig die Architektur dieses Workshops.

3.3. Wann kein interaktives Element?

Drei Situationen, in denen Interaktivität schadet:

- **Kognitive Überforderung.** Wenn die Lernenden gleichzeitig neuen Stoff verarbeiten *und* die Bedienung des Widgets lernen müssen, kostet das Aufmerksamkeit, die für den Inhalt fehlt (Mayer, 2021). Lieber erst Stoff, dann Übung.
- **Reine Illustration.** Wenn das Diagramm keine Variation erlaubt, die didaktisch interessant ist, reicht ein PNG. Interaktivität sollte einen Lernzweck haben, nicht nur „können wir technisch“.
- **Ablenkung durch Kontrolle.** Wenn ein Spiel die Lernenden in den Spielmodus zieht („wie viele Punkte krieg ich?“), kann das vom Inhalt wegführen. Faustregel: Belohnung für **richtige Inhalte**, nicht für Schnelligkeit.

3.4. Mach mit — Lernspiel zur Selbstprüfung

Drei Beispiele, drei Prinzipien, fünf Minuten. Ziehe jedes Beispiel auf das Prinzip, das es illustriert.

💡 Lernspiel öffnen

[Lernspiel: Welches Prinzip steckt hinter welchem Beispiel? →](#)

⚠️ Was, wenn ...

- **Das Spiel öffnet sich nicht?** → Direkt im Browser über die Datei `interactions/lernspiel-kapitel1.html` öffnen. Die Datei ist eine einzige HTML-Datei, ohne Backend.
- **Die Lösungen scheinen mehrdeutig?** → Stimmt — in der Praxis überschneiden sich die Prinzipien. Schau in die Erklärung am Ende des Spiels, dort steht, warum die jeweilige Zuordnung am stärksten passt.

3.5. Tutor

Tutor öffnen

[Tutor „Interaktive Skripte“](#) →

Frag den Tutor nach passenden interaktiven Elementen für dein Kapitel. Vorschlag-Prompt:

Ich überlege, welche interaktiven Elemente in mein Skript zum Thema [Strang A/B/C] passen. Frag mich nach dem Lernziel des jeweiligen Kapitels und schlage dann pro Kapitel EIN passendes Element vor — verbunden mit einem der drei Lernprinzipien (generativ, retrieval, multimedia). Begründe in einem Satz, warum genau dieses Element passt.

4. 2 — Bausteine und Workflow

4.0.0.1. Was du nach diesem Kapitel kannst

- die sechs Bausteine **Quarto**, **YAML**, **JSON**, **HTML**, **iFrame**, **GitHub** in eigenen Worten erklären,
- ein leeres Quarto-Projekt **lokal anlegen** (RStudio + Terminal),
- es mit **GitHub Desktop** in ein neues Repository legen,
- es einmal über **GitHub Pages** veröffentlichen — ein erster klickbarer Link.

4.1. Worum es im Kern geht

Du schreibst Text in einer Datei. Ein Programm verwandelt diese Datei in eine Webseite. Eine zweite Datei legt fest, wie sie aussieht. Ein Online-Speicher liefert die Webseite weltweit aus. Mehr ist es nicht — alles, was wir heute machen, sind Variationen dieses Vier-Zeilen-Workflows. Dass Forschungsarbeit von Versionsverwaltung profitiert, ist seit langem belegt ([Bryan, 2018](#)); Quarto bringt diese Logik in die Lehre ([Allaire et al., 2024](#)).

4.2. Die sechs Bausteine — Lay-Erklärung zuerst

Jeder Begriff wird in der Fachsprache schwerer als nötig. Hier zuerst die einfache Idee, dann das Detail.

Baustein	In einem Satz	Was es technisch ist
Quarto	Programm, das aus einer Textdatei eine Webseite, ein PDF oder Folien macht.	Open-Source-Publishing-System, Nachfolger von R Markdown (Allaire et al., 2024).
YAML	Einstellungsdatei am Anfang jedes Kapitels — sagt z. B., ob es ein Inhaltsverzeichnis gibt.	„YAML Ain’t Markup Language“, strukturiertes Format mit Einrückungen.
JSON	Wie YAML, aber mit Klammern. Wird intern für Daten verwendet.	„JavaScript Object Notation“, Standardformat zum Datenaustausch.
HTML	Sprache, in der Webseiten geschrieben sind.	„HyperText Markup Language“, Tags wie <code><h1></code> oder <code><p></code> .
iFrame	Fenster im Fenster — bettet eine fremde Seite in deine ein.	HTML-Tag <code><iframe src=“...“></code> , isoliert vom Rest der Seite.

Baustein	In einem Satz	Was es technisch ist
GitHub	Speicher für Code im Netz, mit eingebautem Webserver (Pages).	Hosting-Plattform auf Basis von Git, gehört Microsoft.

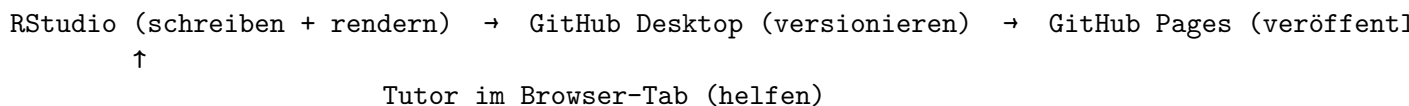
💡 Mach mit — Memory-Spiel zu den Bausteinen

Drei Minuten, sechs Paare. Versuche, alle Begriffe ihrer Lay-Definition zuzuordnen.

[Memory-Spiel öffnen](#) →

4.3. Der Workflow in einem Bild

Drei Werkzeuge, ein Kreislauf:



Versionsverwaltung ist hier kein Nice-to-have. Sie ist die Voraussetzung dafür, dass dein Skript jederzeit reproduzierbar gerendert werden kann und dass du ohne Angst Änderungen vornimmst — ein Punkt, den Bryan (2018) für die Forschungsarbeit ausführlich begründet, der für die Lehre genauso gilt.

4.4. RStudio als Arbeitszentrale — vier Bereiche

Du arbeitest fast immer in RStudio. Vier Bereiche reichen für alles, was wir heute machen:

Bereich	Wo?	Wofür?
Editor	oben links	.qmd-Dateien schreiben
Terminal	unten links, Tab neben Console	<code>quarto render</code> , <code>quarto preview</code>
Files	unten rechts	neue Ordner und Dateien anlegen, Dateien öffnen
Build	oben rechts	Knopf „Render Book“ als Klick-Alternative

Drei nützliche Shortcuts:

- **Strg+Shift+K** — die aktuell geöffnete .qmd-Datei einzeln rendern (für schnelles Vor-schauen einzelner Kapitel).
- **Strg+Shift+B** — komplettes Buch rendern (entspricht dem Build-Knopf).
- **Alt+Shift+R** — neuen Terminal-Tab öffnen, falls keiner sichtbar ist.

Hinweis

Falls dein Terminal-Tab fehlt: **Tools → Terminal → New Terminal**. Voreinstellung auf Windows ist PowerShell, einstellbar unter **Tools → Global Options → Terminal**. Quarto-Befehle funktionieren in jeder Shell.

4.5. Mach mit — die ersten 30 Minuten

Halte dich an diese Reihenfolge. Bei Problemen: Tutor fragen oder zur Trouble-Shooting-Box am Ende springen.

4.5.1. Schritt 1 — neues Projekt in RStudio

1. RStudio öffnen → **File → New Project → New Directory → Quarto Book**.
2. Ordernamen wählen, z. B. `mein-erstes-skript`.
3. Speicherort wählen — am besten direkt in deinem GitHub-Ordner.
4. **Create Project**.

RStudio legt automatisch eine Mini-Buchstruktur an (`_quarto.yml`, `index.qmd`, ein paar Beispielkapitel).

4.5.2. Schritt 2 — Render und Preview (in RStudio)

Im **Terminal-Tab** (unten neben Console):

```
quarto preview
```

Im Browser öffnet sich `http://localhost:4XXX` mit einer Live-Vorschau. Jede Änderung an einer `.qmd`-Datei führt zu sofortigem Reload. Alternative ohne Tippen: **Build-Pane** oben rechts → Knopf „Render Book“ (rendert das ganze Buch einmal, ohne Live-Reload).

4.5.3. Schritt 3 — Beispielkapitel auf deinen Strang anpassen

Öffne `intro.qmd` und ersetze den Beispielinhalt durch eine Mini-Einleitung zu deinem gewählten Strang. Drei Vorlagen, je nach Strangwahl:

4.5.3.1. Strang A — Wissenschaftliche Einstellung

```
# Wissenschaftliche Einstellung
```

```
Wissenschaftliche Einstellung heißt, Aussagen nicht zu glauben, sondern  
zu prüfen - und sich selbst genauso zu prüfen wie andere. Drei Haltungen  
gehören dazu: Skepsis, intellektuelle Demut, Bereitschaft zur Korrektur.  
Quelle: Bergstrom & West (2024).
```

4.5.3.2. Strang B — Recherche und Quellenqualität

Recherche und Quellenqualität

Quellen unterscheiden sich nicht nur darin, ob sie wahr oder falsch sind, sondern darin, wie nachprüfbar sie sind. In diesem Kapitel: drei Filter, mit denen du eine Quelle in 30 Sekunden grob einordnest.
Quelle: Bergstrom & West (2024).

4.5.3.3. Strang C — Transparent schreiben

Transparent schreiben

Transparenz heißt, dass jeder Leser nachvollziehen kann, wie du zu deinen Ergebnissen gekommen bist – welche Daten, welche Methoden, welche Annahmen. In diesem Kapitel: drei Bausteine eines transparenten Methodenteils. Quelle: Bergstrom & West (2024).

4.5.4. Schritt 4 — Ordnerstruktur anlegen (in RStudio)

Im **Files-Pane** (unten rechts) klickst du auf **New Folder** und legst nacheinander an: `images/`, `interactions/`, `include/`, `data/`. Das sind die Ordner für deine späteren Bilder, HTML-Widgets, eingebundene Schnipsel und Datendateien — die Aufteilung folgt der Konvention, die Lakens (2022) in seinem Skript verwendet.

4.5.5. Schritt 5 — GitHub Desktop verknüpfen

1. GitHub Desktop öffnen → **File** → **Add Local Repository** → Ordner wählen.
2. Wenn noch kein Git-Repo: „Create a repository“ akzeptieren.
3. **Publish repository** klicken → Name eingeben (z. B. `mein-erstes-skript`) → **Public** → **Publish**.

Dein Code liegt jetzt auf GitHub.

4.5.6. Schritt 6 — GitHub Pages aktivieren

1. Auf github.com im Repository auf **Settings** → **Pages**.
2. **Source: Deploy from a branch**.
3. **Branch: main** → **Folder: /docs** → **Save**.
4. Nach 1–2 Minuten: dein Skript liegt unter `https://DEIN-USERNAME.github.io/mein-erstes-skript/`.

⚠ Was, wenn ...

- **Render funktioniert lokal, aber GitHub Pages zeigt 404?** → In `_quarto.yml` muss `output-dir: docs` stehen, und der Ordner `docs/` muss committet sein. Schau in `.gitignore`, ob `/docs/` ausgeschlossen wird, und nimm die Zeile heraus.
- **YAML-Fehler beim Rendern?** → Einrückungen prüfen (zwei Leerzeichen, keine Tabs). Doppelpunkte und Anführungszeichen ebenfalls.

- **GitHub Desktop fragt nach Login?** → Im Browser anmelden, dann in GitHub Desktop „Sign in“.
- **quarto: command not found?** → Quarto-Installation prüfen mit `quarto --version`. Wenn leer: neu installieren von <https://quarto.org/docs/get-started/>.

4.6. Tutor

Tutor öffnen

[Tutor „Interaktive Skripte“](#) →

Vorschlag-Prompt für dieses Kapitel:

Ich versuche gerade, ein Quarto-Buch lokal in RStudio anzulegen und über GitHub Pages zu veröffentlichen. Ich bin Anfänger, kein IT-Experte. Erkläre mir Schritt für Schritt, was ich als Nächstes tun muss, immer mit einer Lay-Erklärung VOR dem Code-Schnipsel. Mein aktueller Stand: [hier kurz beschreiben]. Mein Ziel: einen klickbaren Link auf mein Skript. Frag mich zurück, wenn etwas unklar ist.

5. 3a — Inhaltsverzeichnis und Literatur

5.0.0.1. Was du nach diesem Kapitel kannst

- ein **automatisches Inhaltsverzeichnis** in deinem Skript einschalten,
- eine **Zotero-Bibliothek als BibTeX exportieren** und ins Projekt einbinden,
- mit `[@key]` zitieren und ein **automatisches Literaturverzeichnis** am Kapitelende erzeugen.

5.1. Worum es im Kern geht

Ein gutes Skript orientiert die Lesenden zwei Mal: oben durch das Inhaltsverzeichnis, unten durch die Literaturliste. Beides automatisch zu erzeugen ist mehr als Komfort — es vermeidet die typischen Fehlerquellen (vergessene Quellen, falsche Seitenzahlen, doppelte Einträge), die in Hand-pflege unweigerlich entstehen. Die Verbindung von Zotero und Quarto regelt das mit zwei Konfigurationszeilen.

5.2. Inhaltsverzeichnis — drei Zeilen

In `_quarto.yml`:

```
format:
  html:
    toc: true
    toc-depth: 3
    toc-title: "Inhalt"
```

Quarto baut das TOC automatisch aus den Überschriften. Tiefe drei heißt: Kapitel, Abschnitt, Unterabschnitt — alles, was tiefer geschachtelt ist, taucht im TOC nicht auf.

5.3. Zotero anbinden

Zotero exportiert deine Bibliothek als `.bib`-Datei, Quarto liest sie ein, du zitierst per Schlüssel. Das **Better-BibTeX**-Plugin sorgt dafür, dass die Schlüssel stabil bleiben (z. B. `mayer2021`) und der Export sich automatisch aktualisiert, wenn du in Zotero etwas änderst.

5.3.1. Mach mit — in fünf Schritten

1. **Better BibTeX** in Zotero installieren: <https://retorque.re/zotero-better-bibtex/>.
2. Eine kleine **Zotero-Sammlung** anlegen mit zwei oder drei Quellen aus deinem Strang.
3. Sammlung **rechtsklicken** → **Export Collection** → **Better BibTeX** → **Keep up-dated**.
4. Datei speichern als `references.bib` im Projektordner.
5. In `_quarto.yml`:

```
bibliography: references.bib
csl: https://www.zotero.org/styles/apa
link-citations: true
```

5.3.2. Zitieren

Im Text:

```
Generative Lernaktivitäten verbessern das Behalten [@fiorella2016].
Die zugrundeliegende Theorie geht auf Mayer [-@mayer2021] zurück.
```

Quarto rendert das in APA und hängt am Kapitelende automatisch eine Literaturliste an, sofern du dort einen `# Literatur`-Heading mit `::: {#refs}` setzt — wie in diesem Skript.

Mach mit — strangbezogen

Wähle aus deinem Strang **drei Quellen**, lege sie in Zotero an, exportiere und zitiere sie in deinem Kapitel. Wenn du keinen passenden Startpunkt hast, hier je eine Empfehlung:

- **Strang A — Wissenschaftliche Einstellung:** Bergstrom & West (2024), dazu zwei eigene Ergänzungen.
- **Strang B — Recherche und Quellenqualität:** Bergstrom & West (2024), dazu eine Quelle zu Open-Access-Datenbanken (z. B. <https://doaj.org>) und eine zu Pre-Print-Servern.
- **Strang C — Transparent schreiben:** Bergstrom & West (2024), dazu eine Quelle zur TOP-Guideline-Initiative und eine zu Reproducible Research.

Was, wenn ...

- „Citation key not found“? → BibTeX-Schlüssel im `.bib` prüfen, oder Better BibTeX neu auto-exportieren.
- **Liste erscheint nicht?** → Am Kapitelende `# Literatur` und darunter `::: {#refs}` mit `:::` einfügen. Quarto füllt diesen Block.
- **Stil falsch?** → CSL-URL prüfen oder eine eigene CSL-Datei aus <https://www.zotero.org/styles> herunterladen und lokal einbinden (`csl: my-style.csl`).
- **Mehrere Schlüssel zugleich?** → `[@fiorella2016; @mayer2021]` mit Semikolon trennen.

5.4. Tutor

 Tutor öffnen

[Tutor „Interaktive Skripte“](#) →

Vorschlag-Prompt für dieses Kapitel:

Ich möchte aus Zotero exportierte Quellen in einem Quarto-Skript zitieren.
Mein Stand: [Better BibTeX installiert / nicht installiert]; [references.bib liegt
im Projekt / fehlt noch]. Erkläre mir den nächsten Schritt mit Lay-Erklärung
vor dem Code. Frag, wo ich gerade stehe.

6. 3b — TH-Köln-Stil einbauen

6.0.0.1. Was du nach diesem Kapitel kannst

- die Hochschulfarben über `_brand.yml` setzen,
- per CSS scharfkantige Callouts, Tabellen und Code-Blöcke gestalten,
- das TH-Köln-Logo einbinden und prüfen, dass es überall korrekt erscheint.

6.1. Worum es im Kern geht

Gestaltung ist nicht Dekoration, sondern Lese-Hilfe. Ein konsistentes Layout reduziert kognitive Last ([Mayer, 2021](#)) — die Lernenden müssen nicht für jedes neue Element wieder verstehen, was wo wichtig ist. Tufte ([2001](#)) formuliert dasselbe Prinzip für Datenvisualisierung: maximaler Informationsgehalt bei minimalem Tinte-zu-Daten-Verhältnis. Beides spricht für Zurückhaltung im Stil — wenige Farben, klare Hierarchien, kein Schmuck.

6.2. Wie Quarto Aussehen handhabt

Drei Stellschrauben, in dieser Reihenfolge:

1. `_brand.yml` — die Hochschulfarben und -schriften an einem Ort.
2. `styles.scss` — alles, was `_brand.yml` nicht abdeckt (Layout, Akzente, eigene Boxen).
3. `format.html.theme` in `_quarto.yml` — wählt das Bootstrap-Basistheme.

6.3. Die TH-Köln-Palette in Stichworten

Drei Farben, sonst nur Grautöne. Alles andere ist Verstoß gegen das Corporate Design.

Rolle	Hex	Wo?
TH Red	<code>#c81e0f</code>	Links, Akzente, wichtige Hervorhebungen
TH Magenta	<code>#b43092</code>	Strukturlinien, Callout-Ränder
TH Orange	<code>#ea5a00</code>	Sehr sparsam — Warnungen, Verlauf
Ink	<code>#111111</code>	Fließtext (weicherer Schwarz)
Paper	<code>#ffffff</code>	Hintergründe, immer

6.4. Mach mit — _brand.yml kopieren

Im Repository des Workshops liegt eine fertige `_brand.yml` mit der TH-Köln-Palette. Kopiere sie in dein Projektverzeichnis. Beim nächsten `quarto render` sind Links rot, Überschriften schwarz, Akzente magenta.

```
color:
  palette:
    th-red: "#c81e0f"
    th-magenta: "#b43092"
    th-orange: "#ea5a00"
  primary: th-red
  secondary: th-magenta
```

6.5. SCSS-Ergänzungen

Drei Dinge, die `_brand.yml` nicht regelt: scharfe Ecken („kantig“), die magenta H2-Unterstreichungen und die TH-Farbleiste am Seitenanfang. Quarto verarbeitet eigene Stile als **SCSS** (Sass) — eine Erweiterung von CSS, die mit Bootstrap zusammenarbeitet. Du brauchst dafür kein Sass-Wissen: jede gültige CSS-Regel ist auch gültiges SCSS.

⚠️ Warnung

Die Datei muss `styles.scss` heißen (nicht `.css`) und am Anfang einen **Layer-Marker** enthalten — sonst bricht `quarto render` mit einer Fehlermeldung „doesn't contain at least one layer boundary“ ab.

Lege `styles.scss` an und binde sie in `_quarto.yml` ein:

```
format:
  html:
    theme:
      - cosmo
      - styles.scss
```

In `styles.scss` (Auszug — Layer-Marker am Anfang ist Pflicht):

```
/*-- scss:rules --*/

:root {
  --th-red: #c81e0f;
  --th-magenta: #b43092;
}

h1, h2, h3 { border-radius: 0 !important; font-weight: 600; }
h2 { border-bottom: 2px solid rgba(180, 48, 146, 0.35); padding-bottom: 0.3rem; }
.callout { border-radius: 0 !important; }
pre, code { border-radius: 0 !important; }
```

💡 Mach mit — Vorher/Nachher

Render dein Skript einmal **vor** und einmal **nach** dem Einbau von `_brand.yml`. Vergleich beide Versionen nebeneinander. Was ändert sich? Was nicht? Schreib dir drei Beobachtungen auf — wir nutzen sie im Showcase am Ende.

6.6. Logo einbinden

1. Lade das TH-Köln-Logo (PNG, transparent) in `images/th-koeln-logo.png`.
2. In `_brand.yml`:

```
logo:  
  small: images/th-koeln-logo.png  
  medium: images/th-koeln-logo.png
```

3. Render und prüfen.

⚠️ Was, wenn ...

- **Farben tauchen nicht auf?** → Cache leeren (`quarto render --cache-refresh`) oder den Ordner `_freeze/` löschen.
- **CSS wird nicht geladen?** → Pfad in `_quarto.yml` prüfen, relative Pfade ohne führenden Slash.
- **Logo wird nicht angezeigt?** → PNG-Datei muss tatsächlich im `images/`-Ordner liegen, nicht nur in `_brand.yml` referenziert sein.
- **Farben sehen ausgewaschen aus?** → Eventuell ein Bootstrap-Thema aktiv, das die Variablen überschreibt. `theme: [cosmo, styles.scss]` in dieser Reihenfolge halten — die SCSS-Datei muss zuletzt geladen werden.
- **„doesn't contain at least one layer boundary“** beim Render? → Am Anfang von `styles.scss` muss `/*-- scss:rules --*/` stehen.

6.7. Tutor

💡 Tutor öffnen

[Tutor „Interaktive Skripte“](#) →

Vorschlag-Prompt für dieses Kapitel:

Ich möchte mein Quarto-Skript im TH-Köln-Stil gestalten (rot `#c81e0f`, magenta `#b43092`, orange `#ea5a00`, kantige Ecken, Arial). Mein Stand: `[_brand.yml vorhanden / fehlt]`; `[styles.scss vorhanden / fehlt]`. Was ist mein nächster Schritt? Lay-Erklärung zuerst, dann ein konkreter Code-Schnipsel.

7. 4 — Interaktive Diagramme

7.0.0.1. Was du nach diesem Kapitel kannst

- ein **interaktives Diagramm** mit echten Daten in dein Skript einbauen,
- Daten direkt aus dem Netz laden (z. B. **Our World in Data**),
- erklären, **wann** sich Interaktivität didaktisch lohnt und wann nicht.

7.1. Worum es im Kern geht

Diagramme tun in einem Skript zwei Dinge auf einmal — sie zeigen Daten und sie laden zur Interpretation ein. Tufte (2001) formuliert die Maxime: maximale Information bei minimalem Schmuck. Healy (2018) ergänzt für die digitale Lehre: Interaktivität rechtfertigt sich nur, wenn die Lernenden tatsächlich etwas mit ihr anfangen — verschieben, ein- und ausblenden, vergleichen. Andernfalls reicht ein PNG.

7.2. Drei Wege, ein Diagramm einzubauen

Weg	Werkzeug	Wann passend?
R + ggplot2 + plotly	R-Code-Chunk	Du kennst R oder willst es lernen. Volle Kontrolle, schöne Grafiken (Wickham et al., 2019).
Observable JS	OJS-Chunk in Quarto	Du willst dich auf JavaScript einlassen. Sehr schnell, modern.
iFrame mit Datawrapper / Flourish	externer Editor, Embed-Code	Du willst gar nicht programmieren.

Wir gehen heute den ersten Weg — er ist mit RStudio am robustesten und nutzt die R-Pakete, die du wahrscheinlich ohnehin im Werkzeugkasten haben willst.

7.3. Mach mit — ein interaktives Diagramm in 10 Minuten

Wir laden CO -Daten von Our World in Data (Roser et al., 2024) und zeichnen sie als interaktive Linie. Drei Strang-Varianten — jede passt auf das Stoffgebiet, das du gewählt hast.

7.3.0.1. Strang A — Wissenschaftliche Einstellung

Frage: *Wie haben sich Schätzungen zur globalen Lebenserwartung über die Zeit verändert?* Diese Frage zeigt, dass „Fakten“ historisch sind — sie verändern sich mit besserer Datenlage.

```
```{r}
#| label: fig-le
#| fig-cap: "Lebenserwartung bei Geburt, ausgewählte Länder. Daten: Our World in Data (Roser)"
#| message: false

library(plotly)
library(dplyr)
library(readr)

owid <- read_csv("https://ourworldindata.org/grapher/life-expectancy.csv?v=1")

ausgewaehlt <- owid |>
 filter(Entity %in% c("Germany", "Japan", "World"),
 Year >= 1950)

plot_ly(ausgewaehlt,
 x = ~Year, y = ~`Life expectancy at birth (historical)`,
 color = ~Entity, type = "scatter", mode = "lines",
 colors = c("#c81e0f", "#b43092", "#ea5a00")) |>
 layout(yaxis = list(title = "Lebenserwartung (Jahre)"),
 xaxis = list(title = ""))
```
```

7.3.0.2. Strang B — Recherche und Quellenqualität

Frage: *Wie viele wissenschaftliche Publikationen erscheinen pro Jahr?* Antwort führt zur Notwendigkeit, Quellen zu filtern.

```
```{r}
#| label: fig-pub
#| fig-cap: "Wissenschaftliche Publikationen weltweit. Daten: Our World in Data."
#| message: false

library(plotly)
library(dplyr)
library(readr)

owid <- read_csv("https://ourworldindata.org/grapher/scientific-publications.csv?v=1")

plot_ly(owid |> filter(Year >= 1990),
 x = ~Year, y = ~`Articles`,
 color = ~Entity, type = "scatter", mode = "lines",
 colors = c("#c81e0f", "#b43092", "#ea5a00")) |>
 layout(yaxis = list(title = "Publikationen pro Jahr"),
 xaxis = list(title = ""))
```
```

7.3.0.3. Strang C — Transparent schreiben

Frage: *Wie hat sich der Anteil registrierter Studien (Pre-Registration) entwickelt?* Datenquelle hier ggf. selbst recherchieren — Beispiel-Code unten lädt einen kleinen lokalen CSV.

```
```{r}
#| label: fig-prereg
#| fig-cap: "Anzahl pre-registrierter Studien auf OSF. Daten: eigener Auszug."
#| message: false

library(plotly)
library(readr)

prereg <- read_csv("data/osf-preregistrations.csv")

plot_ly(prereg, x = ~Jahr, y = ~Anzahl, type = "bar",
 marker = list(color = "#c81e0f")) |>
 layout(yaxis = list(title = "Pre-Registrierungen"),
 xaxis = list(title = ""))
```
```

Im Browser kannst du jetzt mit der Maus über die Linien fahren, Länder ein- und ausschalten, in Bereiche zoomen. Das Bild im Skript ist live.

7.4. Wann kein interaktives Diagramm?

Drei Situationen, in denen ein statisches PNG genauso gut wirkt:

- **Reine Illustration** — wenn die Lernenden nichts ändern sollen, sondern nur etwas „sehen“, reicht ein Bild ([Tufte, 2001](#)).
- **Print-Version** wichtig — interaktive Plots werden im PDF zu statischen Screenshots; wenn das PDF die Hauptform ist, plane direkt für Print.
- **Performance** — sehr große Datensätze (>100.000 Punkte) machen plotly träge. Dann besser aggregieren oder ein statisches ggplot exportieren.

Mach mit

Tausche im Code zwei Länder gegen welche aus, die für deinen Strang interessant sind. Render, schau das Ergebnis im Browser an, push.

Was, wenn ...

- **could not find function "plot_ly"** → `install.packages("plotly")` in der R-Konsole.
- **CSV lässt sich nicht laden** → Datei lokal in `data/` speichern und von dort lesen (CORS-Beschränkungen). Die OWID-Grapher-URLs ändern sich gelegentlich; im Zweifel den CSV-Download direkt von der Grapher-Seite holen.
- **Diagramm zu klein** → `fig-width: 8` und `fig-height: 5` im Chunk-Header setzen.
- **Spaltenname unbekannt** → `colnames(owid)` in der R-Konsole laufen lassen, um

die exakte Schreibweise zu sehen (manche Spalten haben Sonderzeichen).

7.5. Tutor

Tutor öffnen

[Tutor „Interaktive Skripte“](#) →

Vorschlag-Prompt für dieses Kapitel:

Ich will ein interaktives Diagramm in mein Quarto-Skript einbauen. Daten: [URL oder lokale Datei]. Frage: [welche Variablen, welcher Vergleich]. Ich bin R-Anfänger. Schreibe mir einen kommentierten Code-Chunk, in dem jede Zeile mit einer Lay-Erklärung versehen ist. Falls die Daten nicht ladbar sind, schlag einen Workaround vor.

8. 5 — iFrames, Spiele und KI-Tutor

8.0.0.1. Was du nach diesem Kapitel kannst

- ein **interaktives Quiz oder Spiel** als iFrame in dein Skript einbinden,
- den Unterschied zwischen statischem HTML, **H5P** und externer Einbettung benennen,
- einen **KI-Tutor** als Link einbauen, der die Lernenden begleitet — didaktisch begründet.

8.1. Worum es im Kern geht

Aktivität schlägt Konsum. Lernende, die zwischendurch etwas tun — antworten, ziehen, ausprobieren — behalten mehr und verstehen besser ([Fiorella & Mayer, 2016](#); [Karpicke & Blunt, 2011](#)). Interaktive Elemente sind das Werkzeug, das diese Aktivität in einem statischen Skript ermöglicht. Bei generativer KI als Tutor kommt eine zweite Chance hinzu: ein adaptiver Gesprächspartner, der auf die jeweilige Verständnisstufe reagiert ([Kasneci et al., 2023](#); [Mollick & Mollick, 2023](#)).

8.2. iFrame in einer Zeile

Ein iFrame bettet eine fremde Seite in deine ein:

```
<iframe src="https://h5p.org/h5p/embed/617" width="100%" height="500" frameborder="0"></iframe>
```

Höhe nach Inhalt anpassen, Breite immer 100 %, `frameborder="0"` für ein sauberes Aussehen.

8.3. Drei Wege, ein Spiel einzubauen

Variante	Aufwand	Vorteil	Nachteil
A — Statisches HTML (eigenes Widget)	hoch zur Erstellung, niedrig danach	volle Kontrolle, kein Konto, läuft offline	erfordert HTML/JS-Kenntnisse oder KI-Hilfe
B — H5P (h5p.org / Moodle / WordPress)	mittel	viele fertige Vorlagen (Quiz, Drag-Drop, Memory)	erfordert Konto, externe Abhängigkeit
C — externer Editor (Genially, Kahoot, Quizizz)	niedrig	schnell, hübsch, kollaborativ	Werbung, externe Datenhaltung, oft Login nötig

💡 Mach mit (Variante A — statisches HTML)

Schau dir das **Lernspiel aus Kapitel 1** (interactions/lernspiel-kapitel1.html) im Browser an. Öffne die Datei in RStudio (im **Files**-Tab anklicken → Open in editor) — sie ist eine einzige HTML-Datei, ohne Backend. Diesen Bauplan kannst du selbst (oder dein KI-Tutor) für ein eigenes Spiel im gewählten Strang nutzen.

Strang-Beispiele:

- Strang A — Drag-and-Drop: „Welche Aussage ist eine Hypothese, welche eine Behauptung?“
- Strang B — Multiple Choice: „Welche der drei Quellen ist primärwissenschaftlich?“
- Strang C — Memory: „Methodenbegriff zu transparenter Beschreibung“

💡 Mach mit (Variante B — H5P, optional)

Lege ein kostenloses Konto auf <https://h5p.org> an. Wähle **Drag the Words**, baue ein Mini-Quiz zu drei Begriffen aus deinem Strang. Klicke „Embed“, kopiere den iFrame-Code, füge ihn in dein Kapitel ein. Render.

8.4. KI-Tutor als Link

Der Tutor läuft **nicht** im iFrame, sondern in einem zweiten Tab. Du legst einen GPT (oder Custom GPT) mit System-Prompt an, holst dir den öffentlichen Link, baust ihn als Markdown-Link an passende Stellen im Skript ein. Drei Hosting-Varianten zur Auswahl:

8.4.1. Option 1 — Custom GPT auf ChatGPT (heute genutzt)

Einfachste Variante, weltweit verfügbar, kein Hochschul-Login nötig. Im Skript einbauen:

```
[Frag den Skript-Tutor →] (https://chatgpt.com/g/g-69fec93348108191af1ffb8ce0cf6712-tutor-int)
```

Der vorbereitete Tutor für diesen Workshop liegt unter dem oben verlinkten URL — der System-Prompt steckt dort schon drin. Die Lernenden klicken, ChatGPT-Login öffnet sich (kostenlos), der Tutor antwortet sofort im Workshop-Kontext.

i Hinweis

Datenschutz: ChatGPT speichert Konversationen auf US-Servern. Für offene Lehre ohne personenbezogene Daten unkritisch. Wenn du europäische Datenhaltung brauchst, nimm Option 2.

8.4.2. Option 2 — Custom GPT in der AcademicCloud

Variante mit europäischer Datenhaltung und Hochschul-Login. Du legst einen GPT mit dem vorbereiteten System-Prompt an (siehe [Anhang Tutor-Prompt](#)) und verlinkst ihn:

```
[Frag den Skript-Tutor →] (https://chat.academiccloud.de/chat/DEIN-LINK){target="_blank"}
```

Lernende loggen sich mit Hochschul-Credentials ein.

8.4.3. Option 3 — eigenes statisches Widget oder voll gehosteter Tutor

Für Fortgeschrittene: ein HTML-Widget, das eine OpenAI-kompatible API direkt anspricht (Vorbild: `_relevance-widget.qmd` in <https://github.com/TH-Koln-Bartnik/genai4teaching>), oder eine eigene Web-App auf AWS Amplify mit RAG über Skriptinhalte (Vorbild: [BWL1 Tutor-Chatbot](#)). Mollick und Mollick (2023) geben einen Überblick über die didaktischen Designentscheidungen.

💡 Mach mit

Wir nutzen heute den **vorbereiteten Workshop-Tutor** auf ChatGPT — du musst keinen eigenen GPT bauen, sondern nur den Link einbauen:

```
[Frag den Skript-Tutor →] (https://chatgpt.com/g/g-69fec93348108191af1ffb8ce0cf6712-tutor-i
```

Setze diesen Link an passende Stelle in dein Skript (z. B. in einen Callout-Tip am Kapitelende). Render und prüf den Klick.

Wer einen **eigenen** Tutor mit eigenem System-Prompt bauen will, springt zum [Tutor-Prompt-Anhang](#) und legt sich Option 1 oder Option 2 selbst an.

8.5. Wann kein KI-Tutor?

Drei Situationen, in denen du es lassen solltest:

- **Klar definierte Faktenfragen** — wenn die Antwort eindeutig ist und in einem Lehrbuch steht, ist ein Quiz besser.
- **Sensible persönliche Themen** — der Tutor soll Lernen begleiten, keine Beratung ersetzen.
- **Zeitkritisches Pflichtwissen** — wenn die Lernenden in zehn Minuten etwas können müssen, ist Frontalanleitung effizienter als ein Gespräch ([Kasneci et al., 2023](#)).

⚠️ Was, wenn ...

- **iFrame zeigt nichts an?** → URL prüfen, oft fehlt `https://`. Manche Seiten erlauben kein Embedding (HTTP-Header `X-Frame-Options`) — dann nur als Link verwenden.
- **iFrame zu kurz, Inhalt abgeschnitten?** → `height` höher setzen, oder `style="height: 90vh"` für 90 % der Bildschirmhöhe.
- **AcademicCloud-Link funktioniert für andere nicht?** → Sharing aktivieren in den GPT-Einstellungen, „öffentlich“ oder „mit Link“.
- **Tutor antwortet auf alles, auch außerhalb des Themas?** → System-Prompt nachschärfen, explizit Grenzen setzen („Antworte nur zu Quarto-Workflow-Fragen“).

8.6. Tutor

 Tutor öffnen

[Tutor „Interaktive Skripte“](#) →

Vorschlag-Prompt für dieses Kapitel:

Ich will [ein iFrame-Quiz / einen KI-Tutor-Link / ein eigenes HTML-Widget] in mein Quarto-Skript einbauen. Mein Stand: [...]. Schlag mir den einfachsten ersten Schritt vor und schreib den Code-Schnipsel dazu. Erkläre vorher in zwei Sätzen, was ich da eigentlich tue.

9. 6 — Slides mit Quarto (optional)

i Optional, für Fortgeschrittene

Dieses Kapitel ist nicht Teil der gemeinsamen Übung. Wer mit Kapitel 1–5 früher fertig ist, kann hier weiterbauen. Vorlage und Inspiration: Békés (2025), [Course slides for “Data Analysis with AI”](#).

9.0.0.1. Was du nach diesem Kapitel kannst

- aus einem .qmd-Kapitel einen **Foliensatz** mit Quarto Reveal.js erzeugen,
- die **TH-Köln-Farben** auf Slides übertragen,
- den Foliensatz neben das Skript auf GitHub Pages stellen.

9.1. Schnellbau in vier Schritten

1. Neue Datei `slides/kapitel-1-slides.qmd` anlegen.
2. YAML-Header:

```
---
title: "Kapitel 1 - Didaktik"
author: "Dein Name"
format:
  revealjs:
    theme: [default, ../styles.scss]
    slide-number: true
    chalkboard: true
    incremental: true
    logo: ../images/th-koeln-logo.png
---
```

3. Inhalt: eine **H2-Überschrift** = eine **Folie**, dazwischen Stichpunkte und Bilder. Speaker Notes mit `::: notes`.
4. `quarto render slides/kapitel-1-slides.qmd` → Slide-Deck als HTML.

9.2. TH-Köln-Brand auf Slides

Die `_brand.yml` aus deinem Skript wird auch von Reveal.js gelesen. Zusätzlich einbinden:


```
format:
  revealjs:
    brand: ../_brand.yml
    theme: [default, ../styles.scss]
```

9.3. Tutor

 Tutor öffnen

[Tutor „Interaktive Skripte“ →](#)

Vorschlag-Prompt für dieses Kapitel:

Ich will aus meinem bestehenden Quarto-Kapitel einen Foliensatz mit Reveal.js machen. TH-Köln-Brand soll erhalten bleiben. Schlag mir die einfachste Aufteilung vor: Welche Abschnitte werden eine Folie, welche mehrere? Gib mir den YAML-Header.

Teil II.

Teil 2 — Quick-Starter

10. Quick-Starter Guide

Von Null auf erstem Skript live im Netz — alles aus RStudio

10.0.0.1. Was dieser Anhang leistet

Eine **Schritt-für-Schritt-Anleitung für komplette Anfänger**, parallel zum Workshop nutzbar. Wer alles installiert hat und einmal pushen will: 30 Minuten reine Klickarbeit. **RStudio ist dein einziges Editor-Programm** — Terminal, Files-Pane, Build-Pane und Console liegen alle dort.

10.1. Checkliste vor dem Workshop

Schritt	Wo?	Zeit
Quarto installieren	https://quarto.org/docs/get-started/	5 min
R installieren	https://cran.r-project.org/	5 min
RStudio Desktop (kostenlos)	https://posit.co/download/rstudio-desktop/	5 min
GitHub Desktop	https://desktop.github.com/	5 min
GitHub-Konto erstellen	https://github.com/signup	5 min
Zotero + Better BibTeX	https://www.zotero.org/	10 min
ChatGPT-Konto und Tutor-Link testen	https://chatgpt.com/g/g-69fec93348108191af1ffb8ce0cf6712-tutor-interaktive-skripte	2 min

Insgesamt rund 40 Minuten. Wer schon hat, was er braucht, springt weiter.

10.2. RStudio-Cockpit auf einen Blick

Bereich	Position	Wofür im Workshop?
Editor	oben links	.qmd-Dateien schreiben
Console	unten links, Tab 1	R-Pakete installieren (<code>install.packages(...)</code>)
Terminal	unten links, Tab 2	<code>quarto render</code> , <code>quarto preview</code>
Files	unten rechts, Tab 1	Ordner und Dateien anlegen, Dateien öffnen
Build	oben rechts, Tab 5	Knopf „Render Book“ als Klick-Alternative

Drei Shortcuts, die du dir merken solltest: **Strg+Shift+K** rendert die aktuelle Datei, **Strg+Shift+B** rendert das ganze Buch, **Alt+Shift+R** öffnet einen neuen Terminal-Tab.

10.3. Schritt-für-Schritt: dein erstes Skript

10.3.1. 1. Neues Quarto-Buch anlegen

In RStudio: **File** → **New Project** → **New Directory** → **Quarto Book** → **Create Project**.

10.3.2. 2. Erste Vorschau

Im **Terminal-Tab** (unten links, neben Console):

```
quarto preview
```

Browser öffnet sich. Du siehst die Beispieldateien. **Strg+C** im Terminal beendet den Preview-Server.

10.3.3. 3. _quarto.yml minimal anpassen

Im **Editor** öffnest du `_quarto.yml` und ersetzt den Inhalt durch:

```
project:
  type: book
  output-dir: docs

book:
  title: "Mein erstes Skript"
  chapters:
    - index.qmd
    - intro.qmd

format:
  html:
    theme: cosmo
    toc: true
```

output-dir: docs ist wichtig — GitHub Pages erwartet das später.

10.3.4. 4. Ordner für Bilder, Daten und Widgets anlegen

Im **Files-Pane** (unten rechts) klickst du auf **New Folder** und legst nacheinander an: `images/`, `interactions/`, `include/`, `data/`. So weißt du immer, wo etwas hingehört.

10.3.5. 5. Erstes Kapitel schreiben

Öffne `index.qmd` im Editor, ersetze den Beispieldinhalt durch:

```
# Willkommen

Mein erstes Skript zum Thema [Strang A/B/C].

## Was ich hier behandle

- Punkt 1
- Punkt 2
- Punkt 3
```

10.3.6. 6. Render

Im Terminal:

```
quarto render
```

Oder Knopf **Build** → **Render Book** klicken. Im `docs/`-Ordner liegt jetzt die fertige Webseite.

10.3.7. 7. GitHub Desktop: Repo anlegen

GitHub Desktop → **File** → **Add Local Repository** → Projektordner wählen → **Create a repository** zustimmen → Name: `mein-erstes-skript` → **Publish repository** (Public).

10.3.8. 8. GitHub Pages aktivieren

Auf <https://github.com> im Repo: **Settings** → **Pages** → **Source: Deploy from a branch** → **Branch: main**, **Folder: /docs** → **Save**.

Nach 1–2 Minuten ist dein Skript unter `https://DEIN-USERNAME.github.io/mein-erstes-skript/live`.

10.3.9. 9. Erste Änderung publizieren

Bearbeite `index.qmd`, render lokal (Terminal: `quarto render` oder Knopf), in GitHub Desktop **Commit** → **Push origin**. Webseite aktualisiert sich automatisch in 30–60 Sekunden.

10.4. Was als Nächstes?

Springe zurück in [Kapitel 3a — Übersicht](#) und baue Inhaltsverzeichnis und Literaturverwaltung ein. Dann zu [Kapitel 3b — Stil](#).

11. Trouble-Shooting

Die zehn häufigsten Probleme und wie du sie löst

Jeder Punkt: **Symptom** → **Ursache** → **Lösung**. Wenn nichts hilft: Tutor fragen mit dem Prompt am Ende.

11.1. 1. YAML-Fehler beim Rendern

Symptom: Error: Failed to load YAML metadata. **Ursache:** Einrückung mit Tabs statt Leerzeichen, oder fehlender Doppelpunkt. **Lösung:** YAML-Block prüfen, immer **zwei Leerzeichen** einrücken, keine Tabs. Online-Validator: <https://www.yamllint.com>.

11.2. 2. GitHub Pages zeigt 404

Symptom: Dein Skript ist online, aber die URL liefert „404 — File not found“. **Ursache:** output-dir: docs fehlt in _quarto.yml, oder der Pages-Branch ist falsch eingestellt. **Lösung:** In _quarto.yml output-dir: docs setzen, render, push. In **Settings** → **Pages**: Branch main, Folder /docs.

11.3. 3. docs/ wird nicht committet

Symptom: Nach Push fehlt der docs/-Ordner auf GitHub. **Ursache:** .gitignore schließt /docs/ aus. **Lösung:** Zeile /docs/ aus .gitignore entfernen, dann commit und push.

11.4. 4. Zotero-Zitat erscheint als [?]

Symptom: Zitate werden mit Fragezeichen statt Autor und Jahr gerendert. **Ursache:** Citation Key in der .bib-Datei stimmt nicht mit dem Key im Text überein. **Lösung:** references.bib öffnen, Schlüssel nachsehen, im Text exakt so verwenden. Better BibTeX in Zotero exportiert stabile Keys (z. B. mayer2021).

11.5. 5. iFrame ist leer

Symptom: Der iFrame-Bereich ist sichtbar, aber leer oder zeigt eine Fehlermeldung. **Ursache:** Die externe Seite verbietet Einbettung (HTTP-Header X-Frame-Options). **Lösung:** Statt iFrame einen normalen Markdown-Link verwenden, oder eine andere Quelle wählen.

11.6. 6. CSV lässt sich nicht laden

Symptom: `read_csv()` wirft `CORS error` oder `Forbidden`. **Ursache:** Der Server der Datenquelle erlaubt keinen Cross-Origin-Zugriff aus dem Browser. **Lösung:** Datei lokal in `data/` speichern und von dort laden.

11.7. 7. R-Paket fehlt

Symptom: `Error in library(plotly): there is no package called 'plotly'`. **Ursache:** Paket nicht installiert. **Lösung:** In der R-Konsole `install.packages("plotly")`. Bei mehreren: `install.packages(c("plotly", "dplyr", "readr"))`.

11.8. 8. Quarto-Version zu alt

Symptom: Neue Features (z. B. `_brand.yml`) funktionieren nicht. **Ursache:** Lokale Quarto-Installation ist älter als 1.5. **Lösung:** `quarto --version` prüfen, von <https://quarto.org/docs/get-started/> aktualisieren.

11.9. 9. GitHub Desktop fragt immer nach Anmeldung

Symptom: Bei jedem Push wird das Passwort erneut verlangt. **Ursache:** Personal Access Token läuft ab oder Kontostatus inkonsistent. **Lösung:** In GitHub Desktop **File** → **Options** → **Accounts** → **Sign Out**, dann neu anmelden via Browser.

11.10. 10. Logo wird nicht angezeigt

Symptom: Im gerenderten Skript erscheint kein Logo. **Ursache:** Pfad in `_brand.yml` stimmt nicht, oder Datei fehlt im `images/-`Ordner. **Lösung:** Pfad relativ zur Projektwurzel (`images/th-koeln-logo.png`), Datei dort tatsächlich ablegen.

11.11. Wenn nichts hilft — Tutor fragen

 Tutor öffnen

[Tutor „Interaktive Skripte“](#) →

Vorschlag-Prompt:

Ich versuche, ein Quarto-Skript zu rendern. Folgender Fehler tritt auf:

[Fehlermeldung kopieren]

Mein Setup: [Betriebssystem], Quarto-Version [X.X.X], R-Version [X.X.X], RStudio-Version [X.X.X]. Ich habe schon versucht: [...].

Schreib mir die wahrscheinlichste Ursache in einem Satz, dann den nächsten Schritt zum Beheben. Wenn du mehr wissen musst, frag.

12. KI-Tutor: Link und System-Prompt

Der Workshop-Tutor und die Anleitung zum eigenen Tutor

12.0.0.1. Was dieser Anhang leistet

Du findest hier den **fertigen Tutor** für diesen Workshop (ein Klick) und den vollständigen **System-Prompt**, mit dem du dir denselben Tutor in der AcademicCloud, im eigenen ChatGPT-Account oder als statisches Widget nachbauen kannst.

12.1. Der vorbereitete Workshop-Tutor (sofort nutzbar)

💡 Tutor öffnen

Tutor „Interaktive Skripte“ →

Ein Klick — der GPT öffnet sich in ChatGPT. Login mit kostenlosem ChatGPT-Konto.
Der System-Prompt ist eingebaut.

So baust du den Link in dein eigenes Skript ein:

```
[Frag den Skript-Tutor →] (https://chatgpt.com/g/g-69fec93348108191af1ffb8ce0cf6712-tutor-int
```

In jedem Kapitel als Callout am Ende:

```
::: {.callout-tip}
## Tutor öffnen
[Tutor „Interaktive Skripte“ →] (https://chatgpt.com/g/g-69fec93348108191af1ffb8ce0cf6712-tut
:::
```

12.2. Eigenen Tutor anlegen — drei Hosting-Wege

12.2.1. Weg 1 — eigenes Custom GPT auf ChatGPT

1. <https://chatgpt.com> öffnen, oben links „Explore GPTs“ → „+ Create“.
2. **Configure**-Tab öffnen, Name vergeben, System-Prompt unten kopieren und einfügen.
3. **Save** → **Share** → „Anyone with the link“ oder „Public“.
4. Sharing-Link in dein Skript einbauen (siehe oben).

12.2.2. Weg 2 — AcademicCloud (Hochschul-Login, EU-Datenhaltung)

1. <https://chat.academiccloud.de> öffnen, einloggen.
2. Neuen **Custom GPT** anlegen (Modell: aktuelles Top-Modell).
3. System-Prompt einfügen.
4. **Sharing aktivieren** (öffentlich oder „mit Link“).
5. Link einbauen.

12.2.3. Weg 3 — statisches Widget (nur für Fortgeschrittene)

Eine HTML-Datei mit eingebautem Prompt, die direkt eine OpenAI-kompatible API anspricht — Vorbild: `_relevance-widget.qmd` in <https://github.com/TH-Koln-Bartnik/genai4teaching>. Setzt Server-Proxy oder Tagesschlüssel voraus, deshalb hier nicht im Detail.

12.3. System-Prompt (Copy-Paste)

Du bist ein geduldiger Quarto-Coach für Hochschullehrende ohne IT-Vorkenntnisse. Du begleitest Teilnehmende eines eintägigen Workshops, in dem sie ein eigenes interaktives Skript mit Quarto, RStudio, GitHub Desktop und GitHub Pages erstellen. Beispielthema ihres Skripts: eine kurze Einführung ins wissenschaftliche Arbeiten (Wissenschaftliche Einstellung, Recherche und Quellenqualität, oder Transparent Schreiben).

DEIN VERHALTEN

- Beginne jede Antwort mit einer Lay-Erklärung in ein bis zwei Sätzen. Was ist das, warum gerade jetzt? Erst danach kommt der technische Schritt.
- Frage zu Beginn: "Wo stehst du gerade?" Wenn du keinen Stand kennst, frage nach einer kurzen Beschreibung.
- Gib nie mehr als ZEHN Zeilen Code auf einmal. Lieber kleinere Schritte.
- Erkläre alle Pfade explizit (relativer Pfad, absoluter Pfad, was bedeutet das).
- Verwende deutsche Sprache. Bei Fachbegriffen: einmalig auf Englisch in Klammern, dann auf Deutsch weiter.
- Lasse Fehler nicht klein erscheinen. Wenn etwas typisch falsch läuft, benenne es offen und erkläre warum.

WORKSHOP-INHALT, AUF DEN DU DICH BEZIEHST

- Werkzeuge: Quarto (Version 1.5+), RStudio Desktop, GitHub Desktop, Zotero mit Better BibTeX, optional H5P.
- Stil: TH-Köln-Brand (rot #c81e0f, magenta #b43092, orange #ea5a00), scharfe Ecken, Arial.
- Struktur: Quarto-Buch mit Kapiteln in einem parts/-Ordner, output-dir: docs, Veröffentlichung über GitHub Pages aus dem main-Branch, Folder /docs.
- Inhaltsbeispiel: Einführung wissenschaftliches Arbeiten, drei Stränge: Wissenschaftliche Einstellung / Recherche und Quellenqualität / Transparent Schreiben (basierend auf Bergstrom & West, "Calling Bullshit").

DEINE GRENZEN

- Wechsle nicht auf andere Statische-Seiten-Generatoren (Hugo, Jekyll). Bleib bei Quarto.

- Antworte nicht zu allgemeinen IT-Fragen außerhalb des Skript-Workflows.
- Behaupte nichts, was du nicht sicher weißt. Bei Unsicherheit: sag das.
- Erfinde keine Quellen, keine Pakete, keine API-Endpunkte.

ZIEL

Am Ende des Workshops haben die Teilnehmenden ein eigenes, im Netz veröffentlichtes Skript mit Inhaltsverzeichnis, Zotero-Zitaten, TH-Köln-Stil, einem interaktiven Diagramm und einem iFrame-Element. Hilf ihnen, dorthin zu kommen - Schritt für Schritt.

12.4. Drei Kurz-Prompts für typische Aufgaben

12.4.1. A — Fehler erklären

Erkläre mir diesen Fehler aus dem Quarto-Render-Log in einem Satz, dann den wahrscheinlichsten nächsten Schritt:

[Fehlermeldung einfügen]

12.4.2. B — Code reviewen

Ich habe folgenden YAML/Code in meinem Quarto-Skript. Schau, ob Fehler oder Stolperfallen drin sind. Antworte mit drei kurzen Punkten: was funktioniert, was nicht, was würdest du anders machen.

[Code einfügen]

12.4.3. C — Kapitel ergänzen

Schlage mir für mein Kapitel zum Thema [Strang A/B/C] eine ergänzende Übung vor - kurz, hands-on, mit konkretem Code-Snippet zum Einbauen. Zwei bis drei Minuten Bearbeitungszeit für die Studierenden.

13. Literatur

Alle im Skript zitierten und zur Vertiefung empfohlenen Quellen, alphabetisch nach Erstautor, im **APA-7-Format mit deutscher Lokalisierung** (DGPs-konform). Die Liste wird automatisch aus `references.bib` generiert.

- Allaire, J. J., Teague, C., Scheidegger, C., Xie, Y., & Dervieux, C. (2024). *Quarto: An Open-Source Scientific and Technical Publishing System*. <https://quarto.org/>
- Bergstrom, C. T., & West, J. D. (2024). *Calling Bullshit: Wissenschaftliches Arbeiten und kritisches Denken*. <https://publish.obsidian.md/wagp-calling-bullshit/>
- Bryan, J. (2018). Excuse Me, Do You Have a Moment to Talk About Version Control? *The American Statistician*, 72(1), 20–27. <https://doi.org/10.1080/00031305.2017.1399928>
- Fiorella, L., & Mayer, R. E. (2016). Eight Ways to Promote Generative Learning. *Educational Psychology Review*, 28(4), 717–741. <https://doi.org/10.1007/s10648-015-9348-9>
- Healy, K. (2018). *Data Visualization: A Practical Introduction*. Princeton University Press.
- Karpicke, J. D., & Blunt, J. R. (2011). Retrieval Practice Produces More Learning than Elaborative Studying with Concept Mapping. *Science*, 331(6018), 772–775. <https://doi.org/10.1126/science.1199327>
- Kasneci, E., Sessler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günemann, S., Hüllermeier, E., Krusche, S., Kutyniok, G., Michaeli, T., Nerdel, C., Pfeffer, J., Poquet, O., Sailer, M., Schmidt, A., Seidel, T., ... Kasneci, G. (2023). ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education. *Learning and Individual Differences*, 103, 102274. <https://doi.org/10.1016/j.lindif.2023.102274>
- Lakens, D. (2022). *Improving Your Statistical Inferences*. <https://doi.org/10.5281/zenodo.6409077>
- Mayer, R. E. (2021). *Multimedia Learning* (3. Aufl.). Cambridge University Press. <https://doi.org/10.1017/9781316941355>
- Mollick, E. R., & Mollick, L. (2023). Assigning AI: Seven Approaches for Students, with Prompts. *SSRN Working Paper*. <https://doi.org/10.2139/ssrn.4475995>
- Roediger, H. L., & Karpicke, J. D. (2006). Test-Enhanced Learning: Taking Memory Tests Improves Long-Term Retention. *Psychological Science*, 17(3), 249–255. <https://doi.org/10.1111/j.1467-9280.2006.01693.x>
- Roser, M., Ritchie, H., Mathieu, E., & Ortiz-Ospina, E. (2024). *Our World in Data*. <https://ourworldindata.org/>
- Tufte, E. R. (2001). *The Visual Display of Quantitative Information* (2. Aufl.). Graphics Press.
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Grolemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the Tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>